

WEEK 03

Pthreads API & Classic Problems

Theory Pack

Lifecycle, return values, semaphores, Readers-Writers.

*You know how to create threads.
You know how to protect shared data.
This week: use the full API, return results,
and meet the last major primitive.*

Then solve Readers-Writers.

INSIDE

- 01** Thread Lifecycle & Attributes
- 02** Returning Data From Threads
- 03** Semaphores
- 04** Classic Problems: Readers-Writers

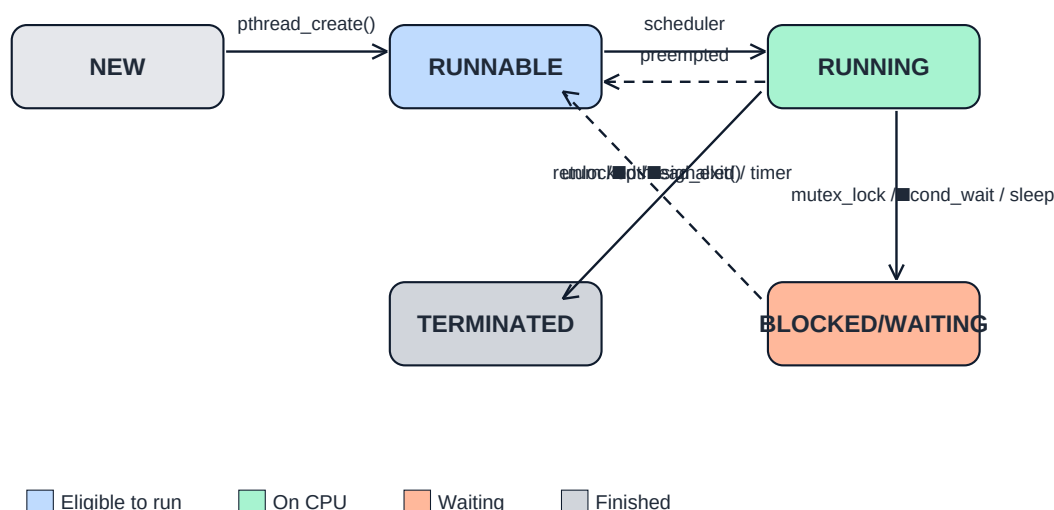
SECTION 01

Thread Lifecycle & Attributes

Every thread you have ever created passed through five states on its way from `pthread_create` to exit. This section names those states, shows the transitions between them, and introduces the configuration API that controls how a thread is created.

The five states

A thread's life can be modelled as a state machine. Understanding the states helps you reason about what a thread is doing at any given moment — and why a program might be stuck.



Thread lifecycle state machine. Solid arrows are driven by your code or the OS scheduler. Dashed arrows are the OS reclaiming the CPU.

State	Meaning	How you get there
NEW	Thread exists in the OS but hasn't executed yet	The moment after <code>pthread_create</code> returns
RUNNABLE	Eligible to run; waiting for a CPU core	From NEW; after unblocking
RUNNING	Actually executing on a CPU core	OS scheduler picks this thread
BLOCKED / WAITING	Waiting for an event, not eligible to run	<code>mutex_lock</code> on a held lock, <code>cond_wait</code> , <code>sleep</code> , I/O
TERMINATED	Thread function has returned or called <code>pthread_exit</code>	return from <code>start_routine</code> ; <code>pthread_exit()</code>

Configuring threads: `pthread_attr_t`

By default, `pthread_create` creates a joinable thread with a default stack size (typically 8 MB on Linux). You can override these defaults using a `pthread_attr_t` object:

```
pthread_attr_t attr;
pthread_attr_init(&attr);                                // initialize with defaults

pthread_attr_setstacksize(&attr, 1024*1024);             // 1 MB stack
pthread_attr_setdetachstate(&attr,
    PTHREAD_CREATE_DETACHED);                             // detached from the start

pthread_create(&t, &attr, worker, arg);                   // use the attributes
pthread_attr_destroy(&attr);                              // clean up the attr object
```

The most important attribute is the detach state — the subject of the next section.

Joinable vs detached: the key choice

Every thread is in one of two modes:

Joinable (the default). When the thread exits, the OS preserves its exit status and a small amount of bookkeeping until another thread calls `pthread_join`. This is analogous to a zombie process that hasn't been waited on. If you never join a joinable thread, those resources are never freed — a resource leak that accumulates over time.

Detached. When the thread exits, the OS immediately reclaims all its resources. Nobody has to (or can) call `pthread_join` on it. Use this for fire-and-forget threads where you don't need the return value and don't want to track every thread you create.

Rule of thumb: if you care about whether the thread succeeded or what it computed, use joinable. If you're spawning background workers that run independently (network connections, log writers), use detached.

Detaching after creation

You can also detach a thread after it's been created — useful when you want to start the thread first and decide later:

```
pthread_t t;
pthread_create(&t, NULL, worker, arg);
pthread_detach(t);    // now t is detached – do not join it
```

Warning: if main returns while detached threads are still running, the entire process is torn down and those threads die mid-execution. Make sure main waits for them using a semaphore or condition variable if their work matters.

SECTION 02

Returning Data From Threads

A thread's start routine returns a `void *`. That pointer can carry a result back to whoever called `pthread_join` — but only if you allocate the result in the right place.

The problem: don't return a stack pointer

The most common Week 3 mistake looks like this:

```
void *worker(void *arg) {
    int result = 42;
    return &result;    // BUG: result dies when this function returns
}
```

The local variable `result` lives on the thread's stack. When the thread returns, its stack is unwound and that memory is gone. The pointer you returned now points to reclaimed memory. Sometimes the old value is still there (the memory hasn't been overwritten yet) and the program appears to work — making this one of the harder bugs to catch in testing.

Approach A — write into the argument struct (recommended)

The cleanest pattern: pass a struct that has both input and output fields. The thread writes its results into the struct, and the caller reads them after joining. No extra heap allocation needed.

```
typedef struct {
    long start, end;    /* inputs */
    long result;        /* output */
} thread_args_t;

void *worker(void *arg) {
    thread_args_t *d = (thread_args_t *)arg;
    d->result = compute(d->start, d->end);    // write output in place
    return NULL;
}

// In main:
thread_args_t args[N];
// ... set args[i].start, args[i].end, create threads ...
// ... join all threads ...
long total = 0;
for (int i=0; i<N; i++) total += args[i].result;    // read results
```

Approach B — malloc the result and return it

If the output type is dynamic or the thread owns the result, allocate it on the heap and return the pointer. `pthread_join` captures it via a void `**` parameter. The caller is responsible for freeing it.

```
void *worker(void *arg) {
    long *result = malloc(sizeof(long));
    *result = compute();    // heap – safe to return
    return result;
}

// In main:
pthread_t t;
pthread_create(&t, NULL, worker, NULL);
long *r = NULL;
pthread_join(t, (void **)&r);    // capture return value
printf("Result: %ld\n", *r);
free(r);    // caller must free
```

When to use which approach? Approach A is simpler and has no extra allocation when the number of threads is fixed. Approach B is better when the result size is not known at creation time, or when the thread allocates a variable-size structure.

Collecting results after joining

You join threads in a separate loop from the creation loop. Joining inside the creation loop (one create, one join, one create, one join...) defeats the purpose — you're back to sequential execution. The pattern is:

```
// Create all threads first
for (int i = 0; i < N; i++)
    pthread_create(&threads[i], NULL, worker, &args[i]);

// Then join them all
for (int i = 0; i < N; i++) {
    pthread_join(threads[i], NULL);
    // now args[i].result (or heap return value) is ready
}
```

SECTION 03

Semaphores

The oldest synchronization primitive. Proposed by Edsger Dijkstra in 1965 — before most modern operating systems existed. A semaphore is simpler than a mutex in some ways and more powerful in others.

What a semaphore is

A semaphore is an integer counter together with two atomic operations:

wait (also called P, down, or decrement): if the counter is greater than 0, decrement it and return immediately. If it's 0, block until someone increments it.

post (also called V, up, or signal): increment the counter. If any threads are blocked waiting, wake one of them.

*The key difference from a mutex: **any thread can post a semaphore**, not just the thread that waited. This makes semaphores ideal for signalling between threads, not just protecting critical sections.*

The POSIX API

```
#include <semaphore.h>

sem_t sem;
sem_init(&sem, 0, initial_value); // 0 = thread-shared (not process-shared)

sem_wait(&sem);    // P: decrement; block if already 0
sem_post(&sem);    // V: increment; wake a waiter if any
sem_trywait(&sem); // non-blocking P; returns EAGAIN if 0

sem_destroy(&sem); // clean up when done
```

Binary vs counting semaphores

When initialized to 1, a semaphore behaves like a mutex — wait acquires it, post releases it. This is a **binary semaphore**. It differs from a real mutex in one important way: a different thread can post it than the one that waited. Some locking conventions rely on this (for example, a thread waiting for a signal it didn't cause).

When initialized to N (greater than 1), the semaphore acts as a **counting semaphore** — it tracks how many of some resource are available. The classic use is limiting concurrent access: initialize to N, each user calls wait before proceeding and post when done, and at most N users run at a time.

Using a semaphore for ordering

The cleanest semaphore use-case: initialize to 0. Thread B calls `sem_wait` and blocks. Thread A does its work, then calls `sem_post`. Now B unblocks and can safely use A's results. No mutex needed — the count of 0 means 'A hasn't finished yet'; the count of 1 means 'A is done.'

```
sem_t done;
sem_init(&done, 0, 0); // start at 0 - B will block

void *thread_a(void *arg) {
    do_work();          // produce something
    sem_post(&done);    // signal: done!
    return NULL;
}

void *thread_b(void *arg) {
    sem_wait(&done);    // wait for A to finish
    use_result();       // safe: A is done
    return NULL;
}
```

Producer-consumer with semaphores

Week 2's producer-consumer used a mutex and two condition variables with while-loop waits. Semaphores express the same logic more directly:

```
sem_t empty; // free slots – initialized to BUFFER_SIZE
sem_t full;  // available items – initialized to 0
pthread_mutex_t mutex; // protects the buffer itself

// Producer:
sem_wait(&empty);           // wait for a free slot
pthread_mutex_lock(&mutex);
buffer[in] = item; in = (in+1) % N;
pthread_mutex_unlock(&mutex);
sem_post(&full);           // item is now available

// Consumer:
sem_wait(&full);            // wait for an item
pthread_mutex_lock(&mutex);
item = buffer[out]; out = (out+1) % N;
pthread_mutex_unlock(&mutex);
sem_post(&empty);          // slot is now free
```

There's no while-loop because `sem_wait` doesn't have spurious wakeups — the counter itself encodes the condition. The semaphore's value *is* the count of available items or free slots; you don't need a separate variable for it.

	Mutex + condvar	Semaphore
Protects critical section	Yes (primary purpose)	Yes (binary mode)
Signalling between threads	Yes (via <code>cond_signal</code>)	Yes (cleaner)
Counts available resources	Requires a separate variable	Built-in (the count IS the resource)
Any thread can signal	No — only lock holder should unlock	Yes
Spurious wakeups possible	Yes — must use while-loop	No

SECTION 04

Classic Problems: Readers-Writers

The Readers-Writers problem models any system where many threads read shared data and a few threads write it — databases, caches, configuration stores, file systems. Solving it correctly introduces a new lock type that you will use for the rest of your career.

The problem

Suppose many threads share a data structure. Some of them only *read* it; some *write* it. The invariants we want to enforce:

- **Multiple readers may read simultaneously** — reads don't modify data, so they can't interfere with each other.
- **A writer needs exclusive access** — while a write is in progress, no readers or other writers may proceed.

A plain `pthread_mutex_t` is too conservative: it allows only one thread at a time, even for concurrent reads that are perfectly safe. In a read-heavy system (say, a web server serving mostly GET requests with occasional config updates), a plain mutex serializes all reads unnecessarily and kills throughput.



Two variants of the problem

Reader preference: a new reader is always allowed in if no writer is currently writing. Writers can starve if readers arrive continuously.

Writer preference: once a writer is waiting, no new readers are admitted. Writers get in as soon as all current readers finish. Readers may wait longer, but writers don't starve.

Neither variant is universally better — it depends on your workload. `pthread_rwlock_t`'s scheduling policy is implementation-defined (and varies between Linux and macOS), so don't rely on either preference in production code without checking.

The solution: `pthread_rwlock_t`

POSIX provides a dedicated reader-writer lock type:

```
pthread_rwlock_t rwlock = PTHREAD_RWLOCK_INITIALIZER;

// Reader:
pthread_rwlock_rdlock(&rwlock); // shared — many threads can hold this
/* ... read shared data ... */
pthread_rwlock_unlock(&rwlock);

// Writer:
pthread_rwlock_wrlock(&rwlock); // exclusive — only one thread at a time
/* ... modify shared data ... */
pthread_rwlock_unlock(&rwlock);

pthread_rwlock_destroy(&rwlock); // when done forever
```


Implementing it manually

For Week 3's problem set you'll implement a reader-writer lock from scratch using only a mutex and a condition variable. The classic reader-preference algorithm uses a reader count plus a write lock:

```
int      reader_count = 0;
pthread_mutex_t count_lock = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t write_lock = PTHREAD_MUTEX_INITIALIZER;

void reader_lock() {
    pthread_mutex_lock(&count_lock);
    reader_count++;
    if (reader_count == 1)          // first reader
        pthread_mutex_lock(&write_lock); // block writers
    pthread_mutex_unlock(&count_lock);
}

void reader_unlock() {
    pthread_mutex_lock(&count_lock);
    reader_count--;
    if (reader_count == 0)          // last reader
        pthread_mutex_unlock(&write_lock); // let writers in
    pthread_mutex_unlock(&count_lock);
}

void writer_lock() { pthread_mutex_lock(&write_lock); }
void writer_unlock() { pthread_mutex_unlock(&write_lock); }
```

*This implements **reader preference**: writers may starve if readers keep arriving. Think about what you'd change to implement writer preference instead — hint: a writer needs to signal to incoming readers that they should wait.*

When to use rwlocks

Reader-writer locks are worth the added complexity only when: (a) reads are much more frequent than writes, (b) critical sections are long enough that the reader overhead doesn't swamp the benefit, and (c) you genuinely need concurrent reads for performance. For short critical sections on lightly contended data, a plain mutex is simpler and often faster because the rlock's internal bookkeeping has overhead too.

***You now have the complete toolkit for the first half of the project.** Mutexes, condition variables, semaphores, and rwlocks give you everything you need to coordinate threads in any pattern you'll encounter. Head to the code examples in `code/`, then tackle `problem-statement.pdf`.*